Command Injection

Trước hết, chúng ta hãy cùng hiểu thế nào là tấn công chèn lệnh. Tấn công chèn lệnh là việc lợi dụng hành vi của một ứng dụng để thực thi các lệnh trên hệ điều hành, sử dụng cùng các đặc quyền mà ứng dụng đó đang chạy trên thiết bị. Ví dụ, tấn công chèn lệnh vào một máy chủ web đang chạy với tư cách người dùng có tên joe sẽ thực thi các lệnh dưới quyền người dùng joe này - và do đó có được bất kỳ quyền hạn nào mà joe có.

Tấn công chèn lệnh (Command injection), hay còn gọi là "Thực thi mã từ xa" (Remote Code Execution - RCE), cho phép thực thi mã từ xa trong một ứng dụng. Những lỗ hổng này thường mang lại lợi nhuận cao nhất cho kẻ tấn công vì nó cho phép kẻ tấn công tương tác trực tiếp với hệ thống dễ bị tổn thương. Ví dụ, kẻ tấn công có thể đọc các tập tin hệ thống hoặc người dùng, dữ liệu và những thứ tương tự.

Ví dụ, việc có thể lợi dụng một ứng dụng để thực hiện lệnh `whoami` nhằm liệt kê tài khoản người dùng mà ứng dụng đang chạy là một ví dụ về tấn công chèn lệnh.

1. Discovering Command Injection

Lỗ hổng này tồn tại vì các ứng dụng thường sử dụng các hàm trong các ngôn ngữ lập trình như PHP, Python và NodeJS để truyền dữ liệu và thực hiện các lệnh gọi hệ thống trên hệ điều hành của máy tính. Ví dụ, lấy dữ liệu nhập từ một trường và tìm kiếm một mục trong một tệp. Hãy xem đoạn mã dưới đây làm ví dụ:

Trong đoạn mã này, ứng dụng lấy dữ liệu mà người dùng nhập vào trường nhập liệu có tên là `$title` để tìm kiếm tiêu đề bài hát trong một thư mục. Chúng ta hãy phân tích điều này thành một vài bước đơn giản.

```
<?php

$songs = "/var/www/html/songs"          1.

if (isset $_GET["title"])) {
    $title = $_GET["title"];            2.

    $command = "grep $title /var/www/html/songtitle.txt";   3.

    $search = exec($command);
    if ($search == "") {
        $return = "<p>The requested song</p><p> $title does </p><b>not</b><p> exist!</p>";
    } else {
        $return = "<p>The requested song</p><p> $title does </p><b>exist!</b>";
      }

    echo $return;                       4.
}

?>
```

1. Ứng dụng lưu trữ các tệp MP3 trong một thư mục nằm trên hệ điều hành.

2. Người dùng nhập tiêu đề bài hát họ muốn tìm kiếm. Ứng dụng lưu trữ thông tin này vào biến `$title`.

3. Dữ liệu trong biến `$title` được truyền đến lệnh `grep` để tìm kiếm trong một tệp văn bản có tên `songtitle.txt` nội dung mà người dùng muốn tìm kiếm.

4. Kết quả tìm kiếm trong `songtitle.txt` sẽ xác định xem ứng dụng có thông báo cho người dùng rằng bài hát đó tồn tại hay không.

Thông thường, loại thông tin này sẽ được lưu trữ trong cơ sở dữ liệu; tuy nhiên, đây chỉ là một ví dụ về việc ứng dụng nhận đầu vào từ người dùng để tương tác với hệ điều hành của ứng dụng.

Kẻ tấn công có thể lạm dụng ứng dụng này bằng cách chèn các lệnh riêng của chúng để ứng dụng thực thi. Thay vì sử dụng `grep` để tìm kiếm một mục trong `songtitle.txt`, chúng có thể yêu cầu ứng dụng đọc dữ liệu từ một tệp nhạy cảm hơn.

Việc lạm dụng ứng dụng theo cách này có thể xảy ra bất kể ngôn ngữ lập trình mà ứng dụng sử dụng. Chỉ cần ứng dụng xử lý và thực thi đoạn mã đó, nó có thể dẫn đến tấn công chèn lệnh. Ví dụ, đoạn mã dưới đây là một ứng dụng được viết bằng Python.

```
import subprocess

from flask import Flask
app = Flask(__name__)                                   1.

def execute_command(shell):
    return subprocess.Popen(shell, shell=True, stdout=subprocess.PIPE).stdout.read()   2.

@app.route('/<shell>')
def command_server(shell):
    return execute_command(shell)                       3.
```

Lưu ý, bạn không cần phải hiểu cú pháp đằng sau các ứng dụng này. Tuy nhiên, để dễ hiểu hơn, tôi cũng đã phác thảo các bước hoạt động của ứng dụng Python này.

- Gói "flask" được sử dụng để thiết lập máy chủ web.
- Một hàm sử dụng gói "subprocess" để thực thi một lệnh trên thiết bị.
- Chúng ta sử dụng một route trong máy chủ web để thực thi bất cứ thứ gì được cung cấp. Ví dụ, để thực thi lệnh whoami, chúng ta cần truy cập http://flaskapp.thm/whoami

Answer the questions below

What variable stores the user's input in the PHP code snippet in this task?

$title — ✓ Correct Answer

What HTTP method is used to retrieve data submitted by a user in the PHP code snippet?

GET — ✓ Correct Answer

If I wanted to execute the `id` command in the Python code snippet, what route would I need to visit?

/id — ✓ Correct Answer

2. Exploiting Command Injection

Chúng ta thường có thể xác định xem có thể xảy ra tấn công chèn lệnh hay không bằng cách quan sát hành vi của ứng dụng, như bạn sẽ thấy trong phần thực hành của phòng học này.

Các ứng dụng sử dụng dữ liệu đầu vào từ người dùng để điền vào các lệnh hệ thống thường có thể được kết hợp theo cách không mong muốn. Ví dụ, các toán tử shell ;, & và && sẽ kết hợp hai (hoặc nhiều) lệnh hệ thống và thực thi cả hai.

Tấn công chèn lệnh có thể được phát hiện chủ yếu theo một trong hai cách:

- Chèn lệnh mù (Blind)
- Chèn lệnh chi tiết (Verbose)

Method	Description
Blind	Kiểu tấn công này là khi không có phản hồi trực tiếp nào từ ứng dụng trong quá trình kiểm thử payload. Bạn sẽ phải điều tra hành vi của ứng dụng để xác định xem payload của bạn có thành công hay không.
Verbose	Kiểu tấn công này cho phép phản hồi trực tiếp từ ứng dụng sau khi bạn đã kiểm tra payload. Ví dụ, chạy lệnh whoami để xem ứng dụng đang chạy dưới tài khoản người dùng nào. Ứng dụng web sẽ hiển thị tên người dùng trực tiếp trên trang.

Detecting Blind Command Injection

Tấn công chèn lệnh mù là khi việc chèn lệnh xảy ra; tuy nhiên, không có đầu ra nào hiển thị, vì vậy nó không dễ dàng bị phát hiện ngay lập tức. Ví dụ, một lệnh được thực thi, nhưng ứng dụng web không xuất ra bất kỳ thông báo nào.

Đối với loại tấn công chèn lệnh này, chúng ta cần sử dụng các payload gây ra độ trễ thời gian. Ví dụ, các lệnh ping và sleep là những payload quan trọng để kiểm tra. Sử dụng lệnh ping làm ví dụ, ứng dụng sẽ bị treo trong x giây tương ứng với số lần ping bạn đã chỉ định.

Một phương pháp khác để phát hiện tấn công chèn lệnh mù là bằng cách buộc xuất ra một số đầu ra. Điều này có thể được thực hiện bằng cách sử dụng các toán tử chuyển hướng như >. Ví dụ, chúng ta có thể yêu cầu ứng dụng web thực thi các lệnh như whoami và chuyển hướng kết quả đến một tệp. Sau đó, chúng ta có thể sử dụng một lệnh như cat để đọc nội dung của tệp mới được tạo này.

Kiểm tra tấn công chèn lệnh theo cách này thường phức tạp và đòi hỏi khá nhiều thử nghiệm, đặc biệt là vì cú pháp của các lệnh khác nhau giữa Linux và Windows.

Lệnh `curl` là một cách tuyệt vời để kiểm tra tấn công chèn lệnh. Điều này là bởi vì bạn có thể sử dụng `curl` để truyền dữ liệu đến và từ một ứng dụng trong payload của mình. Hãy xem đoạn mã

dưới đây làm ví dụ, một payload `curl` đơn giản gửi đến một ứng dụng là hoàn toàn có thể thực hiện tấn công chèn lệnh.

```
curl http://vulnerable.app/process.php%3Fsearch%3DThe%20Beatles%3B%20whoami
```

Detecting Verbose Command Injection

Phát hiện tấn công chèn lệnh theo cách này có thể nói là phương pháp dễ nhất trong hai phương pháp. Tấn công chèn lệnh chi tiết là khi ứng dụng cung cấp phản hồi hoặc đầu ra về những gì đang xảy ra hoặc đang được thực thi.

Ví dụ, đầu ra của các lệnh như ping hoặc whoami được hiển thị trực tiếp trên ứng dụng web.

Payload	Description
Whoami	Kiểm tra xem ứng dụng đang chạy dưới tài khoản người dùng nào.
Ls	Liệt kê nội dung của thư mục hiện tại. Bạn có thể tìm thấy các tệp như tệp cấu hình, tệp môi trường (mã thông báo và khóa ứng dụng) và nhiều thứ có giá trị khác.
Ping	Lệnh này sẽ khiến ứng dụng bị treo. Điều này sẽ hữu ích trong việc kiểm tra ứng dụng về khả năng tấn công bằng phương pháp chèn lệnh mù.
Sleep	Đây là một payload hữu ích khác trong việc kiểm tra ứng dụng về tấn công chèn lệnh mù, trong trường hợp máy không cài đặt ping.
nc	Netcat có thể được sử dụng để tạo một shell đảo ngược trên ứng dụng dễ bị tổn thương. Bạn có thể sử dụng điểm khởi đầu này để điều hướng xung quanh máy mục tiêu nhằm truy cập các dịch vụ, tệp tin khác hoặc các phương tiện tiềm năng để leo thang đặc quyền.

Answer the questions below

What payload would I use if I wanted to determine what user the application is running as?

whoami — ✓ Correct Answer

What popular network tool would I use to test for blind command injection on a **Linux** machine?

ping — ✓ Correct Answer

What payload would I use to test a **Windows** machine for blind command injection?

timeout — ✓ Correct Answer

3. Remediating Command Injection

Tấn công chèn lệnh có thể được ngăn chặn bằng nhiều cách khác nhau. Từ việc hạn chế tối đa việc sử dụng các hàm hoặc thư viện có khả năng gây nguy hiểm trong ngôn ngữ lập trình đến việc lọc đầu vào mà không cần dựa vào thao tác của người dùng. Tôi đã trình bày chi tiết hơn về những điều

này bên dưới. Các ví dụ bên dưới là của ngôn ngữ lập trình PHP; tuy nhiên, các nguyên tắc tương tự có thể được mở rộng cho nhiều ngôn ngữ khác.

Vulnerable Functions

Trong PHP, nhiều hàm tương tác với hệ điều hành để thực thi các lệnh thông qua shell; bao gồm:

- Exec
- Passthru
- System

Hãy xem đoạn mã dưới đây làm ví dụ. Ở đây, ứng dụng chỉ chấp nhận và xử lý các số được nhập vào biểu mẫu. Điều này có nghĩa là bất kỳ lệnh nào như "whoami" sẽ không được xử lý.

```
<input type="text" id="ping" name="ping" pattern="[0-9]+"></input>  1.
<?php
echo passthru("/bin/ping -c 4 ".$_GET["ping"]."); 2.

?>
```

- Ứng dụng chỉ chấp nhận một mẫu ký tự cụ thể (các chữ số từ 0-9).
- Sau đó, ứng dụng sẽ chỉ tiến hành xử lý dữ liệu này, toàn bộ dữ liệu đều là số.

Input sanitisation

Việc lọc bỏ các dữ liệu đầu vào từ người dùng mà ứng dụng sử dụng là một cách tuyệt vời để ngăn chặn tấn công chèn lệnh. Đây là quá trình xác định định dạng hoặc loại dữ liệu mà người dùng có thể gửi. Ví dụ, một trường nhập liệu chỉ chấp nhận dữ liệu số hoặc loại bỏ bất kỳ ký tự đặc biệt nào như > , & và / .

Trong đoạn mã bên dưới, hàm PHP filter_input (mở trong tab mới) được sử dụng để kiểm tra xem dữ liệu được gửi qua biểu mẫu nhập liệu có phải là số hay không. Nếu không phải là số, đó chắc chắn là dữ liệu đầu vào không hợp lệ.

```
<?php

if (!filter_input(INPUT_GET, "number", FILTER_VALIDATE_NUMBER)) {

}
```

Bypassing Filters

Các ứng dụng sẽ sử dụng nhiều kỹ thuật khác nhau để lọc và làm sạch dữ liệu nhận được từ đầu vào của người dùng. Các bộ lọc này sẽ giới hạn bạn chỉ được truy cập vào các dữ liệu cụ thể; tuy nhiên, chúng ta có thể lợi dụng logic đằng sau ứng dụng để vượt qua các bộ lọc này. Ví dụ, một ứng dụng có thể loại bỏ dấu ngoặc kép; thay vào đó, chúng ta có thể sử dụng giá trị thập lục phân của chúng để đạt được kết quả tương tự.

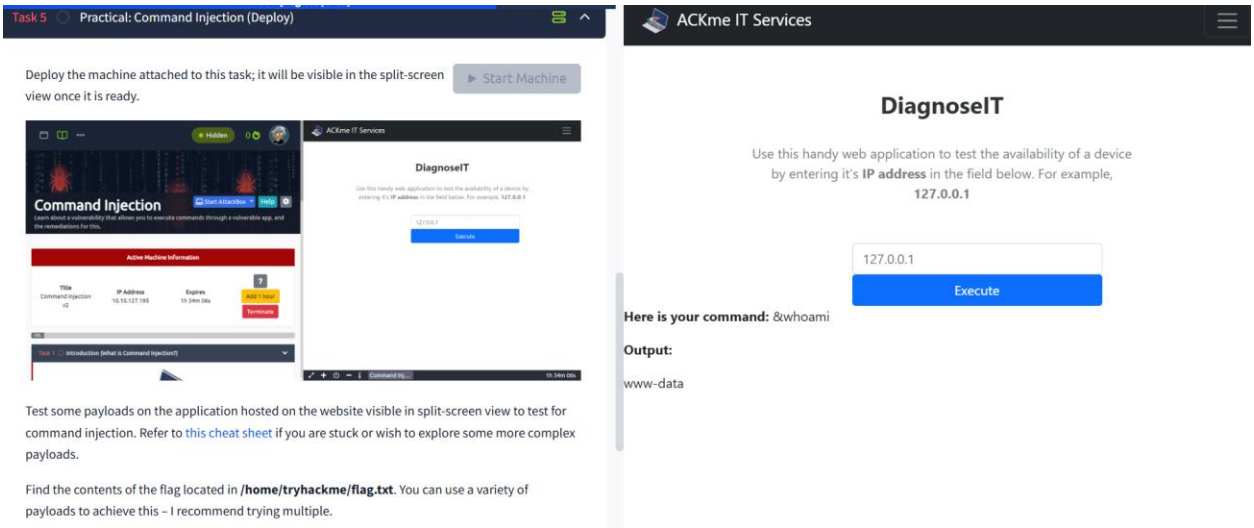
Khi được thực thi, mặc dù dữ liệu được cung cấp sẽ ở định dạng khác với định dạng mong đợi, nhưng nó vẫn có thể được diễn giải và sẽ cho ra kết quả tương tự.

```
$payload = "\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64"
```

What is the term for the process of "cleaning" user input that is provided to an application?

sanitisation ✓ Correct Answer

4. Practical



Trong phần này, tôi sẽ sử dụng máy ảo được dựng sẵn của tryhackme để trả lời các câu hỏi

Question 1:

What user is this application running as?

Tôi sẽ thử sử dụng "&whoami". Kết quả:

Here is your command: &whoami

Output:

www-data

What user is this application running as?

www-data

✓ Correct Answer

Question 2:

What are the contents of the flag located in **/home/tryhackme/flag.txt**?

___{_____}

Check

Tôi sẽ thử sử dụng lệnh "&dir **/home/tryhackme"**, lệnh này nhằm mục đích liệt kê các tệp có trong đường dẫn.

Here is your command: &dir /home/tryhackme

Output:

flag.txt flag.txt.save

Chúng ta thấy tệp flag.txt nằm trong path, ta sử dụng lệnh cat để đọc

Here is your command: &cat /home/tryhackme/flag.txt

Output:

THM{COMMAND_INJECTION_COMPLETE}

Kết quả

What are the contents of the flag located in **/home/tryhackme/flag.txt**?

THM{COMMAND_INJECTION_COMPLETE}

✓ Correct Answer